

Spring Boot Bootcamp Study

프로젝트 보고서

1. 프로젝트 개요

1.1 프로젝트명

Spring Boot Bootcamp Study

1.2 GitHub 저장소

https://github.com/zuzusnasna/bootcamp_study2

1.3 프로젝트 목적

본 프로젝트는 Spring Boot 학습 과정에서 학습한 기술을 단순히 이론으로 기록하는 것이 아니라, 직접 실습 코드를 작성하고 기능을 구현하면서 Java 기반 백엔드 개발의 전체적인 흐름을 이해하는 것을 목표로 진행하였다.

DI, AOP, Lombok과 같은 Spring 핵심 개념부터 JDBC, MyBatis, JPA, Spring MVC, Thymeleaf, Spring Security, 배포까지 단계적으로 학습하였다.

이후 학습한 기술을 실제 프로젝트에 적용하기 위해 회원 인증 및 인가 기능을 포함한 게시판을 구현하였으며, 개발 과정에서 발생한 오류를 직접 분석하고 해결하면서 실무적인 문제 해결 과정도 경험하였다.

2. 개발 환경

개발 언어: Java

프레임워크: Spring Boot

웹 기술: Spring MVC, Thymeleaf, JSP

보안: Spring Security

데이터 접근 기술: JDBC, MyBatis, JPA

데이터베이스: H2

빌드 도구: Gradle

배포: JAR, WAR, Tomcat

환경 설정: Spring Profile

개발 도구: IntelliJ IDEA

형상 관리: Git, GitHub

3. 학습 진행 과정

본 프로젝트는 Spring의 핵심 개념을 시작으로 데이터베이스 접근 기술, 웹 개발, 보안, 배포, 게시판 구현 순서로 학습 범위를 확장하였다.

2026년 9월 3일에는 AOP, DI, Lombok을 학습하였다.

2026년 9월 4일에는 JDBC와 MyBatis를 학습하였다.

2026년 9월 7일에는 JPA와 CRUD 기능을 학습하였다.

2026년 9월 8일에는 JSP, MVC, RESTful API, Thymeleaf, JPA 연관관계를 학습하였다.

2026년 9월 9일에는 회원 수정 기능과 Spring Security의 인증 및 인가, UserDetailsService, BCrypt를 학습하였다.

2026년 9월 10일에는 Spring Boot 배포, JAR/WAR, Spring Profile, H2, REST API, 환경별 설정을 학습하고 게시판 구현을 진행하였다.

4. 주요 학습 내용

4.1 Spring 핵심 기능

Spring Boot의 기본적인 객체 관리와 의존성 관리 구조를 학습하였다.

DI(Dependency Injection), IoC(Inversion of Control), AOP(Aspect Oriented Programming), Lombok, Spring Bean 관리와 Service 및 Repository 계층 구조를 실습하였다.

이를 통해 객체를 개발자가 직접 생성하고 관리하는 방식과 달리 Spring이 IoC Container를 통해 객체의 생명주기와 의존성을 관리하는 구조를 이해하였다.

4.2 JDBC

JDBC를 이용하여 Java 애플리케이션에서 데이터베이스에 접근하는 기본적인 방법을 학습하였다.

데이터베이스 연결, SQL 실행, 결과 조회 등의 과정을 직접 구현하면서 애플리케이션과 데이터베이스가 연결되는 기본적인 동작 원리를 이해하였다.

4.3 MyBatis

MyBatis를 통해 SQL을 직접 관리하면서 데이터베이스에 접근하는 방식을 학습하였다.

JDBC에서 직접 처리해야 했던 반복적인 데이터 접근 작업을 줄이고, SQL과 Java 객체를 연결하는 과정을 실습하였다.

이를 통해 SQL 중심의 데이터 접근 방식과 객체 중심의 데이터 접근 방식의 차이를 이해하였다.

4.4 JPA

JPA를 활용하여 객체 중심으로 데이터를 관리하는 방법을 학습하였다.

주요 학습 내용은 Entity, Repository, CRUD, 연관관계, Spring Data JPA 등이 있다.

JPA Repository를 활용하여 데이터 조회, 저장, 수정, 삭제 기능을 구현하면서 애플리케이션의 객체와 데이터베이스 테이블이 연결되는 구조를 이해하였다.

5. Spring MVC 및 Thymeleaf

Spring MVC를 활용하여 웹 애플리케이션의 요청 처리 구조를 학습하였다.

전체적인 처리 구조는 다음과 같다.

사용자 요청

↓

Controller

↓

Service

↓

Repository

↓

Database

Controller에서는 클라이언트의 요청을 받아 적절한 Service를 호출하고, Service에서는 비

즈니스 로직을 처리한 후 Repository를 통해 데이터를 조회하거나 저장하는 구조를 실습하였다.

Thymeleaf를 이용하여 서버에서 전달된 데이터를 HTML 화면에 출력하고, 회원 및 게시판 기능과 연결하였다.

6. Spring Security

Spring Security를 활용하여 로그인과 인증 및 인가 구조를 구현하였다.

주요 학습 내용은 다음과 같다.

Authentication

Authorization

UserDetails

UserDetailsService

PasswordEncoder

BCrypt

Role 기반 접근 제어

로그인 및 로그아웃

특히 회원 비밀번호를 평문으로 저장하지 않고 BCrypt를 이용하여 암호화하는 방식을 적용하였다.

또한 사용자에게 부여된 권한에 따라 특정 기능에 접근할 수 있도록 Role 기반 접근 제어를 적용하였다.

이를 통해 단순한 로그인 기능뿐만 아니라 인증된 사용자가 어떤 기능에 접근할 수 있는지를 제어하는 인가의 개념까지 학습하였다.

7. 게시판 구현

7.1 게시판 구현 목적

기존에 학습한 Spring Boot 기술을 하나의 프로젝트에 종합적으로 적용하기 위해 게시판을 구현하였다.

Spring Boot, Spring MVC, JPA, Thymeleaf, Spring Security를 활용하여 회원 관리와 게시글 관리 기능을 구현하였다.

단순 CRUD 기능 구현에 그치지 않고 로그인한 사용자와 게시글 작성자를 연결하고, 사용자 권한에 따라 기능 접근을 제한하는 구조까지 구현하였다.

7.2 회원 기능

구현한 회원 관련 기능은 다음과 같다.

회원가입

로그인

로그아웃

회원 정보 수정

비밀번호 변경

BCrypt 기반 비밀번호 암호화

Spring Security 기반 인증

관리자 권한 확인

회원 정보를 데이터베이스에 저장하고 로그인 과정에서 사용자를 조회하여 Spring Security의 인증 과정과 연결하였다.

7.3 게시글 기능

게시판에서는 다음과 같은 기능을 구현하였다.

게시글 목록 조회

게시글 페이징

게시글 상세 조회

게시글 작성

게시글 수정

게시글 삭제

게시글 작성자 확인

로그인 사용자와 게시글 작성자 연결

게시글 작성 및 수정 과정에서는 입력값에 대한 Validation을 적용하였으며 BindingResult를 이용하여 검증 결과를 처리하였다.

7.4 관리자 기능

Spring Security의 Role 기반 권한 관리를 활용하여 관리자 기능을 구현하였다.

ROLE_ADMIN 권한을 가진 사용자는 관리자 기능에 접근할 수 있도록 설정하였다.

일반 사용자가 관리자 전용 기능에 접근하지 못하도록 인증과 권한 검사를 적용하였다.

이를 통해 로그인 여부만 확인하는 인증(Authentication)과 사용자의 권한을 확인하는 인가(Authorization)의 차이를 실제 기능으로 구현하였다.

8. 게시판 전체 처리 구조

게시판의 기본적인 요청 처리 과정은 다음과 같다.

사용자

↓

Thymeleaf 화면

↓

Controller

↓

Service

↓

Repository



JPA



H2 Database

사용자가 게시글 목록이나 상세 페이지를 요청하면 Controller가 요청을 처리하고 Service 계층을 통해 비즈니스 로직을 수행한다.

이후 Repository를 통해 데이터베이스에 접근하고 조회된 결과를 다시 Controller를 통해 화면으로 전달한다.

로그인의 경우 Spring Security가 인증 과정에 개입한다.

로그인 요청



Spring Security



UserDetailsService



회원 조회



BCrypt 비밀번호 검증



Authentication 생성



권한 확인



서비스 접근

이러한 구조를 직접 구현하면서 Spring Boot 애플리케이션에서 인증부터 데이터 처리까

지 이어지는 전체적인 흐름을 이해하였다.

9. 트랜잭션 처리

회원 데이터 삭제와 같이 여러 데이터에 영향을 주는 기능에서는 @Transactional을 활용하였다.

특히 회원 삭제 과정에서 해당 회원이 작성한 게시글을 먼저 처리한 후 회원 데이터를 삭제하는 흐름을 구현하였다.

이를 통해 하나의 작업이 여러 데이터베이스 작업으로 구성될 경우 전체 작업이 하나의 단위로 처리되어야 한다는 트랜잭션의 개념을 학습하였다.

10. 배포 및 환경 설정

Spring Boot 애플리케이션을 개발 환경에서 실행하는 것에 그치지 않고 실제 배포 과정도 학습하였다.

JAR와 WAR의 차이를 학습하고 외부 Tomcat을 이용한 WAR 배포 과정을 실습하였다.

또한 Spring Profile을 이용하여 환경에 따라 설정을 분리하는 방법을 학습하였다.

주요 학습 내용은 다음과 같다.

Spring Boot JAR

Spring Boot WAR

외부 Tomcat

Spring Profile

H2 Database

spring.profiles.active

환경별 설정

이를 통해 개발 환경과 배포 환경의 설정을 분리하고 Spring Boot 애플리케이션이 서버 환경에서 실행되는 과정을 이해하였다.

11. 트러블슈팅 경험

11.1 로그인 및 인증 문제

Spring Security를 적용하면서 사용자 인증 객체와 서비스 계층의 사용자 정보를 연결하는 과정에서 발생하는 문제를 확인하고 수정하였다.

인증 과정에서 사용되는 사용자 정보가 실제 회원 데이터와 어떻게 연결되는지 확인하면서 UserDetailsService의 역할을 이해하였다.

11.2 비밀번호 암호화 문제

BCrypt를 적용하는 과정에서 데이터베이스에 저장된 비밀번호와 로그인 시 입력되는 평문 비밀번호가 직접 비교되는 것이 아니라 BCrypt의 검증 과정을 통해 비교되어야 한다는 점을 확인하였다.

이를 통해 비밀번호 암호화와 인증 과정의 동작 원리를 이해하였다.

11.3 게시물 권한 문제

로그인한 사용자와 게시물 작성자를 비교하여 게시물 수정 및 삭제 권한을 제한하였다.

이를 통해 단순히 로그인 여부만 확인하는 것이 아니라 실제 게시물 작성자 여부를 판단하여 기능 접근을 제한하는 방법을 학습하였다.

11.4 로그아웃 및 접근 경로 문제

로그아웃 이후 페이지 접근과 재로그인 과정에서 발생한 URL 및 Controller 처리 문제를 확인하고 수정하였다.

이를 통해 Spring Security의 인증 상태와 Controller의 요청 경로가 서로 어떻게 연결되는지 확인하였다.

11.5 게시물 삭제 문제

게시물 삭제 기능을 구현하는 과정에서 발생한 문제를 분석하고 Controller, Service, Repository 계층의 처리 과정을 확인하여 삭제 기능이 정상적으로 동작하도록 수정하였다.

12. Git 및 GitHub 활용

프로젝트 전체 과정에서 Git과 GitHub를 이용하여 학습 코드와 개발 과정을 관리하였다.

기능을 구현한 후 바로 커밋하고 푸시하는 방식으로 작업하여 개발 과정을 단계별로 기록하였다.

전체적인 작업 과정은 다음과 같다.

기능 구현

↓

테스트

↓

오류 발생

↓

원인 분석

↓

코드 수정

↓

재테스트

↓

Git Commit

↓

Git Push

↓

문서화

게시판 구현 과정에서도 기능별 수정 사항과 트러블슈팅 내용을 커밋으로 기록하였다.

또한 README와 별도의 학습 문서를 작성하여 단순히 코드만 저장하는 것이 아니라 학습 과정과 구현 내용을 함께 관리하였다.

13. 프로젝트를 통해 습득한 역량

13.1 Backend

Java 기반 백엔드 개발

Spring Boot 구조 이해

MVC 패턴

Controller / Service / Repository 계층 구조

REST API

13.2 Database

JDBC

MyBatis

JPA

CRUD

Entity

Repository

JPA 연관관계

13.3 Security

Spring Security

Authentication

Authorization

UserDetailsService

BCrypt

PasswordEncoder

Role 기반 접근 제어

13.4 Web

Spring MVC

Thymeleaf

JSP

RESTful API

서버와 클라이언트 간 데이터 처리

13.5 Deployment

Spring Boot JAR

WAR

Tomcat

Spring Profile

환경별 설정

13.6 형상관리

Git

GitHub

기능 단위 Commit

Git Push

README 작성

학습 문서 관리

트러블슈팅 기록

14. 프로젝트 성과

본 프로젝트를 통해 Spring Boot의 개별 기능을 단순하게 학습하는 단계에서 벗어나 하나의 웹 애플리케이션을 직접 구성하는 전체적인 백엔드 개발 흐름을 경험하였다.

특히 다음과 같은 개발 과정을 반복적으로 경험하였다.

기술 학습

↓

실습 코드 작성

↓

기능 구현

↓

오류 발생

↓

원인 분석

↓

코드 수정

↓

테스트

↓

Git Commit

↓

문서화

이를 통해 단순히 Spring Boot 문법을 학습하는 것이 아니라 실제 개발 과정에서 발생하는 문제를 직접 분석하고 해결하는 경험을 쌓았다.

또한 Spring MVC, JPA, Thymeleaf, Spring Security를 하나의 게시판에 적용하면서 인증, 인가, 데이터 처리, 화면 구성, 트랜잭션 처리 등 백엔드 애플리케이션의 주요 요소를 직접 경험하였다.

15. 향후 발전 방향

현재 구현한 게시판을 기반으로 다음과 같은 기능을 추가할 수 있다.

댓글 기능

게시글 검색 기능

게시글 카테고리 기능

파일 업로드 기능

게시글 좋아요 기능

관리자 기능 확대

REST API 고도화

예외 처리 및 공통 응답 구조 개선

테스트 코드 확대

실제 운영 데이터베이스 연동

Docker 기반 배포

CI/CD 구축

16. 결론

bootcamp_study2는 Spring Boot를 학습하면서 작성한 코드를 단순하게 보관하는 저장소가 아니라 Spring Boot 백엔드 개발 역량을 단계적으로 쌓아가는 학습 및 실습 프로젝트이다.

DI와 AOP 같은 Spring 핵심 개념부터 JDBC, MyBatis, JPA, Spring MVC, Thymeleaf, Spring Security, 배포까지 학습 범위를 확장하였으며, 최종적으로 게시판 구현을 통해 각 각의 기술을 하나의 애플리케이션에 적용하였다.

특히 GitHub를 이용하여 학습 기록과 기능 구현 과정을 지속적으로 관리함으로써 개발 결과뿐만 아니라 문제 해결 과정과 성장 과정까지 확인할 수 있는 포트폴리오형 학습 저장소로 구성하였다.

이를 통해 Java와 Spring Boot를 기반으로 한 백엔드 개발의 기본적인 구조와 개발 프로세스를 이해하고, 직접 기능을 구현하고 오류를 해결하는 실무적인 개발 경험을 쌓을 수 있었다.

17. 포트폴리오 핵심 요약

Spring Boot의 핵심 기술을 단계적으로 학습하고 Spring MVC, JPA, Thymeleaf, Spring Security를 활용한 게시판 구현까지 진행하였다.

이를 통해 백엔드 애플리케이션의 기본적인 설계부터 CRUD, 데이터베이스 연동, 회원 인증 및 인가, BCrypt 비밀번호 암호화, 권한 관리, 트랜잭션 처리, 배포 및 환경 설정까지 경험하였다.

또한 기능 구현 과정에서 발생한 오류를 직접 분석하고 수정하면서 문제 해결 능력을 향상시켰으며, Git과 GitHub를 활용하여 개발 과정을 기능 단위로 기록하고 관리하는 경험을 쌓았다.